后端代码合规审查报告

审查时间：2026-04-20

审查范围：`backend/` 目录全量（基础设施层 + 业务模块层 + packages）

对照规范：`.github/copilot-instructions.md` + 七份 instructions 文件

总体评级：⚠️ WARN（基础设施层基本合规；业务模块层全空；含 2 个安全漏洞需修复）

违规清单

编号	严重程度	文件路径	违规描述	所违反规范
V001	CRITICAL	`lib/shared/encryption.dart`	AES-CBC 使用固定 IV（`IV.fromLength(16)`），代码注释自注"生产应每次随机"。相同明文始终产生相同密文，不满足语义安全（OWASP A02:2021 加密失效）	security-checklist §4；架构约束第3条
V002	HIGH	`lib/core/middleware/auth_middleware.dart`	`JWT.verify()` 未传 `algorithms: ['HS256']`，未显式排除 `alg: none` 算法混淆攻击	security-checklist §2
V003	HIGH	`lib/modules/*/controllers\`	services\	repositories\
V004	HIGH	`lib/config/database.dart`	`sslMode: SslMode.disable` 禁用 TLS，生产环境 DB 连接明文传输	security-checklist §数据库传输；OWASP A02
V005	MEDIUM	`lib/config/app_config.dart`	`corsOrigins` 默认值为 `'*'`，生产环境若未设 `CORS_ORIGINS` 环境变量则跨域全开放	security-checklist §5
V006	MEDIUM	`lib/router/app_router.dart`	路由表为骨架状态，RBAC 权限矩阵（约 90 条规则）无法被任何业务请求触发	backend-controller.instructions
V007	LOW	`lib/core/middleware/rate_limit_middleware.dart`	直接构建 `Response(429, ...)` 而非抛出 `AppException`，绕过 `error_handler.dart` 统一处理	backend-controller.instructions
V008	LOW	`packages/rent_escalation_engine/lib/src/`	所有递增类型逻辑集中在 `calculator.dart`，未按规范拆分为 `engines/` 子目录下独立文件	dart-package.instructions

---

合规通过项

- ✅ **AppConfig**：6 个必填环境变量缺失时 `throw StateError`，`JWT_SECRET` 和 `ENCRYPTION_KEY` 均做 `length < 32` 强校验

- ✅ **AppException 层级**：6 个快捷子类完整，code 格式为 `SCREAMING_SNAKE_CASE`
- ✅ **error_handler.dart**：`AppException → {"error":{"code":...,"message":...}}`，非预期异常返回 500 且不暴露堆栈，符合信封格式规范
- ✅ **RBAC 双层设计**：Pipeline 级 `rbacMiddleware()` + Per-route `withRbac()` 装饰器，二房东路径隔离（`/api/subleases/portal` 前缀 + `boundContractId` 校验）均已实现
- ✅ **UserRole 枚举**：8 角色完整，`fromString()` 含向后兼容别名（`admin/lease_specialist/read_only`）
- ✅ **PaginatedResult**：`page/pageSize/total` 三字段，`offset` 计算封装，符合分页约定
- ✅ **EncryptionService**：AES-256-CBC 结构正确，`encryptField/decryptField/maskField` 方法齐全，`****后4位` 脱敏逻辑正确
- ✅ **server.dart 管道顺序**：`errorHandler → log → rateLimit → auth → rbac → audit → router`，与规范一致
- ✅ **packages 零外部依赖**：`kpi_scorer` 和 `rent_escalation_engine` 的 `pubspec.yaml` 中 `dependencies: {}` 正确，`publish_to: none`
- ✅ 单元测试存在：`scorer_test.dart` + `calculator_test.dart` 已建立
- ✅ **KPI 打分**：`clamp(0.0, 100.0)` 边界处理，正/反向插值公式实现正确
- ✅ 迁移文件：无裸 `TIMESTAMP`，使用事务 `BEGIN/COMMIT`，枚举扩展在事务外执行（正确处理 PostgreSQL 限制）

修复优先级建议

🔴 CRITICAL / HIGH（阻断上线）

V001 — 固定 IV（CRITICAL）

在 `encryptField` 中每次生成随机 IV，并将其拼入密文头部（`iv_bytes + cipher_bytes` → base64），`decryptField` 先分离 IV 再解密。需同步迁移已存入的数据。

```
// ✅ 修复方案
String encryptField(String plainText) {
  final iv = IV.fromSecureRandom(16); // 每次随机
  final encrypted = _encrypter.encrypt(plainText, iv: iv);
  // 前16字节为 IV，后续为密文
  final combined = iv.bytes + encrypted.bytes;
  return base64Encode(combined);
}

String decryptField(String combined64) {
  final bytes = base64Decode(combined64);
  final iv = IV(Uint8List.fromList(bytes.sublist(0, 16)));
  final cipher = Encrypted(Uint8List.fromList(bytes.sublist(16)));
  return _encrypter.decrypt(cipher, iv: iv);
}
```

V002 — JWT 算法限制（HIGH）

```
// ✅ 修复方案（一行）
final jwt = JWT.verify(token, SecretKey(jwtSecret), algorithms: ['HS256']);
```

V004 — 数据库 SSL（HIGH）

```
// ✅ 修复方案：通过环境变量控制
final sslMode = config.isProduction ? SslMode.verifyFull : SslMode.disable;
settings: PoolSettings(maxConnectionCount: 10, sslMode: sslMode),
```

🟡 HIGH（sprint 内修复）

V003 — 业务模块实现

按 Phase 1 优先级（M1 → M2 → M3 → M4 → M5）逐模块落地四层实现，每个 Controller 路由用 `withRbac` 挂载后在 `app_router.dart` 取消注释。

🟠 MEDIUM（技术债务）

*V005 — CORS `` 默认值**

改为在启动时判断：若 `CORS_ORIGINS == '*'` 且非开发环境，输出 `WARN` 日志提醒；或将该字段改为必填。

V006 — 路由注册

随业务模块实现同步完成，V003 修复后自然消除。

🟢 LOW（优化项）

V007 — rate_limit 直接 Response

```
// ✅ 改为抛出 AppException
throw AppException('RATE_LIMIT_EXCEEDED', '请求过于频繁，请稍后再试', 429);
```

V008 — package 文件拆分

将 `calculator.dart` 中各递增类型的 `_calculate` 分支提取为 `engines/` 目录下独立文件（`fixed_rate_engine.dart`、`fixed_amount_engine.dart`、`stepped_engine.dart` 等）。

总结

当前 backend 的基础设施层（配置 / 鉴权 / RBAC / 错误处理 / 加密 / 分页）架构设计基本对齐规范，质量较高。

主要问题集中在三个方向：

1. **加密安全漏洞**（V001 固定 IV）— 必须在任何数据写入生产之前修复
2. **JWT 算法限制缺失**（V002）— 一行代码修复，无理由延迟

3. **业务模块全空**（V003）— 当前最大进度缺口，是 Phase 1 交付的核心工作量